Sentinelleai — Audit Report

MagnetaOFTStandardFactory.sol · 568a441d-ab54-4265-b87e-212626cd509b · 2026-06-08 10 h 41 min 40 s

CAUTION 62 / 100 9 consolidated findings

MagnetaOFTStandardFactory presents a moderate risk profile. The most significant confirmed finding is the use of single-step Ownable (SC01) rather than Ownable2Step, confirmed by multiple agents and the static sentinel (OWN-1). All privileged operations — treasury management, cross-chain creator wiring, token-ops module wiring, and fee withdrawal — flow through a single EOA owner with no timelock or multisig requirement, creating a key-compromise or fat-finger risk analogous to documented incidents (Drift 2026, IoTeX 2026). The static sentinel's HIGH finding (AST-MISSING-ACCESS-CONTROL) is a false positive: createOFTStandardToken is intentionally public and fee-gated, confirmed by the developer agent and contract documentation. The ARI-1 MEDIUM (division-before-multiplication) is also a false positive — the regex matched comment text, not arithmetic. The ZAD-1 findings for setCrossChainCreator and setTokenOpsModule are intentional design choices (zero-address as disable sentinel), documented in NatDoc; the setTreasury ZAD-1 is already mitigated with an explicit zero-address revert. The FACT-2 finding for createForCreator is also mitigated by an explicit zero-address check. Residual risks include permanently locked ETH beyond accumulatedFees (no rescue path), potential gas griefing via an uncapped external call to tokenOpsModule, and the operational rigidity of a constant fee. No reentrancy, oracle, or flash-loan vectors were identified. The contract is safe to deploy with the caveat that the owner key must be protected by a multisig and Ownable2Step should be adopted to prevent irreversible ownership loss (SC01).

Severity breakdown

1 MEDIUM 3 LOW 5 INFO

Static sentinel pre-scan

Static sentinels (EVM/Solidity) matched 7 pattern(s) — 1 HIGH, 2 MEDIUM, 4 LOW.

MEDIUM SC07 · ARI-1 · line 48

Division before multiplication (precision loss)

(a / b) * c truncates intermediate result. Use mulDiv.

```
/// updateMetadata/enableRevoke* on behalf of the creator (with
```

LOW SC01 · OWN-1 · line 18

Single-step Ownable (no Ownable2Step)

Ownership transferred to a wrong address is irreversible. Ownable2Step requires the new owner to accept, preventing typos and lost keys.

```
contract MagnetaOFTStandardFactory is Ownable
```

LOW SC01 · ZAD-1 · line 94

Setter accepts zero-address (no validation)

set*(address _x) functions that write the parameter directly to state can brick the protocol if called with address(0). Add require(_x != address(0)).

```
function setCrossChainCreator(address _creator) external onlyOwner { emit
CrossChainCreatorUpdated(crossChainCreator, _creator); crossChainCreator = _creator;
}
```

LOW SC01 · ZAD-1 · line 104

Setter accepts zero-address (no validation)

set*(address _x) functions that write the parameter directly to state can brick the protocol if called with address(0). Add require(_x != address(0)).

```
function setTokenOpsModule(address _module) external onlyOwner { emit
TokenOpsModuleUpdated(tokenOpsModule, _module); tokenOpsModule = _module; }
```

LOW SC01 · ZAD-1 · line 217

Setter accepts zero-address (no validation)

set*(address _x) functions that write the parameter directly to state can brick the protocol if called with address(0). Add require(_x != address(0)).

```
function setTreasury(address _newTreasury) external onlyOwner { if (_newTreasury ==
address(0)) revert ZeroAddress(); address old = treasury; treasury = _newTreasury;
emit TreasuryUpdated(old, _newTreasury);
```

MEDIUM SC01 · FACT-2 · line 194

Factory accepts address parameters without zero-address check

create*/deploy* functions that store `address` parameters (token, recipient, manager) without `require(_x != address(0))` brick the resulting pool — token transfers revert, fees go to the zero address, etc.

```
function createForCreator — unchecked: creator
```

HIGH SC01 · AST-MISSING-ACCESS-CONTROL · line 155

External state-mutating function lacks access control

An external/public, non-view function that writes to state without an onlyOwner / role / msg.sender check is callable by anyone. Even if the name doesn't match a privileged-keyword whitelist, the lack of any gate is suspicious.

```
function createOFTStandardToken( string memory name, string memory symbol, string memory tokenURI, uint256 totalSupply, bool revokeUpdate, bool revokeFreeze, bool revokeMint ) exte
```

Consolidated findings

MEDIUM CVSS 5.5 SC01: Access Control

Single-step Ownable — irreversible ownership transfer risk

The contract inherits OpenZeppelin Ownable (single-step) rather than Ownable2Step. All privileged operations (setCrossChainCreator, setTokenOpsModule, setTreasury, withdraw) are gated solely on the owner EOA. A typo, phishing, or key-compromise event that transfers ownership to a wrong address is irreversible. The historian agent escalated this to CRITICAL by analogy to Drift 2026, but the contract itself does not hold user funds at scale (only accumulated create-fees), so the blast radius is limited to fee loss and factory misconfiguration rather than protocol-wide drain. Severity is assessed MEDIUM accordingly.

```
Line 18: contract MagnetaOFTStandardFactory is Ownable
```

→ Replace Ownable with Ownable2Step so the new owner must accept before the transfer finalises. Additionally, consider a multisig owner and a timelock on high-impact setters (setTreasury, setCrossChainCreator).

Confirmed by: static-sentinel (OWN-1), developer, economist, historian

LOW CVSS 3.1 SC02: Asset Management

Ether sent directly to contract beyond accumulatedFees is permanently locked

The withdraw() function only transfers the amount tracked in accumulatedFees. Any ETH arriving via direct transfer, selfdestruct, or other means increments address(this).balance but not accumulatedFees, making it unrecoverable with the current interface.

```
Function withdraw() — no receive/fallback rescue path
```

→ Add an owner-only rescue function that withdraws address(this).balance to treasury, or at minimum document that untracked ETH is intentionally non-recoverable.

Confirmed by: economist

LOW CVSS 2.9 SC02: Asset Management

Uncapped external call to tokenOpsModule enables gas griefing

In _deployAndRegister, the factory makes a raw .call() to tokenOpsModule with no gas cap. A malicious or buggy module set by a compromised owner could consume nearly all remaining gas before failing, forcing token creators to pay for wasted gas. The nonReentrant guard prevents reentrancy but does not bound gas consumption.

```
Lines 147–150: tokenOpsModule.call(abi.encodeWithSelector(0x4a4f0aac,
tokenAddress))
```

→ Forward a fixed gas stipend (e.g., {gas: 50000}) to the external call to cap griefing potential, if bytecode size permits.

Confirmed by: economist

LOW CVSS 2.1 SC01: Access Control

setCrossChainCreator and setTokenOpsModule accept address(0) without explicit guard

Both setters intentionally accept address(0) as a disable sentinel, documented in NatDoc. The static sentinel (ZAD-1) flagged these as missing zero-address validation, but the developer confirmed this is by design and the createForCreator function explicitly checks crossChainCreator != address(0). Risk is low but worth noting for auditors unfamiliar with the sentinel pattern.

```
Lines 94–97 (setCrossChainCreator), Lines 104–107 (setTokenOpsModule)
```

→ No code change required. Consider adding an explicit NatDoc @dev note clarifying that address(0) is a valid disable value to prevent future maintainers from adding an incorrect zero-address guard.

Confirmed by: static-sentinel (ZAD-1 ×2), developer

INFO CVSS 0.0 SC07: Arithmetic

FALSE POSITIVE — Static sentinel ARI-1: division-before-multiplication matched comment text

The static sentinel ARI-1 flagged line 48 for division-before-multiplication. Inspection confirms the matched text is inside a NatDoc comment ('updateMetadata/ enableRevoke* on behalf of the creator (with'), not an arithmetic expression. No precision-loss vulnerability exists.

```
Line 48: NatDoc comment
```

→ No action required. Static rule produced a false positive on comment text.

Confirmed by: developer

INFO CVSS 0.0 SC01: Access Control

FALSE POSITIVE — Static sentinel AST-MISSING-ACCESS-CONTROL on createOFTStandardToken

The static sentinel flagged createOFTStandardToken as lacking access control. This function is the intentional public fee-gated entry point for the factory. It requires msg.value >= createFee, uses nonReentrant, and is the documented primary interface. No access control is appropriate here.

```
Line 155: function createOFTStandardToken
```

→ No action required. The function is correctly public and fee-gated by design.

Confirmed by: developer

INFO CVSS 0.0 SC01: Access Control

FALSE POSITIVE — Static sentinel ZAD-1 on setTreasury already has zero-address guard

The static sentinel flagged setTreasury for missing zero-address validation. The function already contains 'if (_newTreasury == address(0)) revert ZeroAddress();' making this a false positive.

```
Line 217: function setTreasury
```

→ No action required.

Confirmed by: developer

INFO CVSS 0.0 SC01: Access Control

FALSE POSITIVE — Static sentinel FACT-2 on createForCreator already validates creator

The static sentinel flagged createForCreator for not checking the creator parameter. The function contains 'if (creator == address(0)) revert ZeroAddress();' making this a false positive.

```
Lines 194–210: function createForCreator
```

→ No action required.

Confirmed by: developer

INFO CVSS 0.0 SC02: Asset Management

Operational rigidity — constant createFee requires redeployment to change

createFee is a uint256 public constant. Fee adjustments require full factory redeployment and re-pointing of all integrations. This is a documented intentional trade-off to stay under the Spurious Dragon bytecode limit. No security vulnerability, but noted for operational awareness.

```
Line 24: uint256 public constant createFee = 0.01 ether
```

→ Document the redeployment procedure clearly. Consider a mutable fee in a future version if bytecode headroom allows.

Confirmed by: economist

Historical precedents

Retrieved from the local bug-bounty corpus (~20 700 findings) by vulnerability-class match.

CRITICAL `rounding` ethereum 2025-11-03 **$120 000 000**

BalancerV2 — Precision Loss (2025-11-03)

https://x.com/BlockSecTeam/status/1986057732810518640

CRITICAL `rounding` optimism 2024-05-14 **$20 000 000**

Sonne Finance — Precision loss (2024-05-14)

https://neptunemutual.com/blog/taking-a-closer-look-at-sonne-finance-exploit/

HIGH `rounding` optimism 2023-04-15 **$7 000 000**

HundredFinance — Donate Inflation ExchangeRate && Rounding Error (2023-04-15)

https://twitter.com/peckshield/status/1647307128267476992

CRITICAL `access-control` solana 2022-02-02 **$320 000 000**

Bridge — guardian-set signature verification bypass via missing Signer check

https://rekt.news/wormhole-rekt/

CRITICAL `access-control` solana 2026-04-01 **$285 000 000**

Drift Trade — Compromised Admin + Fake Token Price Manipulation

CRITICAL `access-control` solana 2022-03-23 **$48 000 000**

Stablecoin mint — missing has_one between bank and collateral accounts

https://rekt.news/cashio-rekt/

LOW `bm25` 2021-11-01

nested — Missing zero-address check in setters

https://github.com/code-423n4/2021-11-nested-findings/issues/83

MEDIUM `bm25` 2023-07-01

tapioca — MISSING ZERO-ADDRESS CHECK IN CONSTRUCTOR

https://github.com/code-423n4/2023-07-tapioca-findings/issues/81

GAS `bm25` 2021-11-01

nested — Add zero-address checkers

https://github.com/code-423n4/2021-11-nested-findings/issues/108

MEDIUM `bm25` 2022-05-01

rubicon — `RubiconMarket.feeTo` set to zero-address can DoS `buy` function

https://github.com/code-423n4/2022-05-rubicon-findings/issues/319

MEDIUM bm25 2023-04-01

rubicon — `feeTo` set to zero-address can DoS the `rubicall` function

https://github.com/code-423n4/2023-04-rubicon-findings/issues/213

MEDIUM bm25 2023-04-01

frankencoin — `_challengeSeconds ` accepts 0 value when creating a position

https://github.com/code-423n4/2023-04-frankencoin-findings/issues/794

Generated by Sentinelleai on 2026-06-08T14:47:25.461Z. Audit ID: `568a441d-ab54-4265-b87e-212626cd509b`.

This report combines deterministic regex+AST sentinels, optional Slither/Mythril static analysis, and a 6-model adversarial LLM panel consolidated by a Judge agent.